

Open in app ↗

 MediumGet unlimited access to the best of Medium for less than \$1/week. [Become a member](#)

Advanced Graphics Programming in Unreal, part 7

17 min read · Jun 26, 2025



William Mishra-Manning



Listen



Share



More

This is a seven-part article series.

Part 1: [Introduction](#)

Part 2: [Rendering Abstractions \(RHI, RDG\) and Threading](#)

Part 3: [Scenes, Views, and SceneViewExtension](#)

Part 4: [Global Shaders](#)

Part 5: [Simple Material Shaders](#)

Part 6: [Advanced Material Shaders](#)

Part 7: Mesh-Material Shaders and Mesh Passes

Mesh-Material Shaders

In this final article, we will go over mesh passes using Mesh-Material Shaders. This is a variant of Material Shaders which draw specific Primitive Components from the game scene.

NOTE: “Primitive Component” is what Unreal calls any renderable thing in the world

We will add a mesh pass to the demo that draws into the Game of Life state, making it feel more interactive and tied to the world. I will also add a bonus mesh pass to the

Demo which I invite you to explore on your own.

Theory

Vertex Factories

By now you should be comfortable with the idea that Material Shaders compile against every applicable Material, to create different behaviors within the same shader. Mesh-Material shaders add a second layer of compilation, **Vertex Factories**, to handle the huge variety of geometry in a 3D scene.

A Static Mesh, Skeletal Mesh, Niagara Particle System, and Random Voxel Plugin you Bought on the Marketplace all have their own kind of **Vertex Factory**, generating the vertex data that your shader needs:

- Static meshes will pull from vertex buffers.
- Skeletal meshes will pull from dynamic vertex buffers, and/or apply dynamic bone transforms in the vertex shader.
- Niagara particles could have a huge variety of vertex data buffers, depending on the specific effect. They can also choose between GPU- or CPU-based storage.
- Third-party plugins that offer custom primitives, such as voxel landscapes, will sometimes have their own Vertex Factory which you can't plan for.

Creating your own factory is out of scope for this series, but like all things Unreal it doesn't seem too hard once you can get through the boilerplate.

In Unity, for example, you couldn't plan on one way to handle all of the different kinds of primitives. In many cases, like skeletal meshes, it's nearly impossible to get the data you want. But in Unreal you can write your shader to be *completely agnostic* to all these data formats! You only need one shader to work with all of them, by setting it up to compile against the object's **Vertex Factory** and invoking that factory's shader code. It's the exact same technique used to integrate with Materials.

Workflow

Writing a custom mesh pass starts pretty simply, with defining a new Mesh-Material shader class and USF file. Usually you will need at least two shaders, Vertex and

Pixel, but it's common to use one USF file to define both of them.

Unfortunately it doesn't stop there — the primary workflow for Mesh-Material shaders is significantly different than for others. Here is a quick rundown of what makes them special:

RDG usage

The mesh pass should go through the global helper functions `AddSimpleMeshPass(...)` or `AddDrawDynamicMeshPass(...)`. Both of these take a lambda for you to assemble your draw calls within, executed immediately (not during RDG execution).

It's not 100% clear to me what the difference is between the two, but

`AddDrawDynamicMeshPass(...)` is only used for decal rendering in the engine while most other passes seem to use `AddSimpleMeshPass(...)`. The latter takes an `FInstanceCullingManager` while the former does not, yet you can also pass nullptr for the culling manager and things work fine.

Components and their Batches

Any renderable primitive splits its contents into “mesh batches”. A single batch roughly corresponds to a draw call. However you must convert them into real draw calls for your pass, and that means enlisting the help of a “mesh-pass processor” by defining a new child of `FMeshPassProcessor`.

In fact, it's tricky to even get the mesh batches from a primitive in the first place. My library provides a helper to do that, `EGP::ForEachBatch(...)`, which works for all the primitive types tried so far except landscapes — unfortunately I cannot get landscapes working in a mesh pass :(

Actually, back up even further: merely gathering all the renderable components in a scene is arcane and prone to multithreaded crashes! So again, I made a utility to handle this. It's explained in detail below, but the gist is that you create a new type of scene component and attach it to any Primitive Component which you want to send to the render pass. If you need to render most/all scene objects in your pass, I recommend editing the engine source to add a new official pass instead of following

this article.

Parameters

Mesh-Material shaders can't really use shader parameter structs. You may remember back in article 4, Global Shaders, that there is a "Legacy" workflow for parameters? Now you're forced to use that. This is described more in detail below.

Gathering Primitives

In theory, and according to the Unreal docs, it should be possible to iterate over all renderable mesh batches in a Scene-View. However I have never gotten that to work -- the official code snippet iterates over all *decals* in the scene, which is not terribly useful.

Instead I like to define a custom component which attaches to specific Primitive Components, to signal them to our mesh pass. This new component handles the tricky details of getting primitive data into the render thread.

As mentioned before, if you really want a mesh pass that draws most or all of the scene then I highly recommend forking the engine to add it as a new official pass. Not only to get around this problem, but also to get the full performance benefit of static batching which isn't available to a plugin mesh pass. If you go this route then use the existing mesh passes as a guide; the Renderer module contains an enum of passes and a special array holding each one's Mesh Pass Processor.

My library provides a framework for gathering primitives, to go with the Scene-View Extension and Pass object we created back in part 3. Assuming you already have that SVE and Pass object, do the following:

- Create a new child of `U_EGP_RenderPassComponent`, and attach it to other components which should be part of your mesh pass.
- For any per-object parameters you want in your mesh shader, add fields to this new component. I highly recommend exposing them to Blueprints and the Details panel with `(BlueprintReadWrite, EditAnywhere)`.
- Define a POD proxy struct for the component, holding copies of its fields for the

render thread. If you don't have any fields, just define an empty struct.

- Inside the component class, invoke the macro

`EGP_PASS_COMPONENT_SIMPLE_PROXY_IMPL(FMyProxy, expr)` . Here `expr` is an expression that constructs the proxy struct. If creating your proxy struct is complicated, then paste the definition of the macro into your class to get more control.

- Override `GetPassType()` to report the type of your pass object, for example

`u_GOL_RenderPass::StaticClass()` .

- Update the parent type of your scene view extension

(`T_EGP_RenderPassSceneViewExtension`) to have two more template parameters: the new component type, and its proxy struct.

Now your SVE can easily iterate over all marked components in the scene, by calling `ForEachComponent_RenderThread(someLambda)` . Components also have a toggle to remove them from the effect, which makes them stop appearing in this iteration.

Parameters

There is no clear way to use parameter structs in a Mesh-Material shader. You need to fall back to the low-level Legacy method, parameter fields, which was explained in the **Detail** section of [the 4th article \(on Global Shaders\)](#). You can also see it in practice in this article's **Demo** section.

Along with the lack of parameter structs, there is a totally different workflow for passing and binding parameters for a mesh pass. The bright side is this new workflow allows you to have different parameter values per-object if you wish.

1. Define a custom subtype of `FMeshMaterialShaderElementData` . This struct represents the parameters for one mesh batch in your custom pass, and the base type only contains some LOD info. Add fields representing your shader parameters, both per-batch and global. If your pass has no parameters, you can use the parent type directly and skip the next two steps.
2. In your shader class, shadow the built-in member function `GetShaderBindings(...)` . Change the argument of type

`FMeshMaterialShaderElementData` to your new subtype.

3. Inside your `GetShaderBindings(...)` overload, make sure to call the parent's implementation, then bind your own shader parameter fields to the element data that was passed in using `FMeshDrawSingleShaderBindings`. To understand this step better, see the section on [Legacy parameters back in part 4](#).
4. As you write the Mesh Pass Processor for your pass (covered below), you'll be creating one instance of the element data struct for each mesh batch. Here is the standard snippet, found within most implementations of

`FMeshPassProcessor::AddMeshBatch(...)` :

```
FMShaderElementData element;  
//Set up universal element data (culling, LOD stuff).  
elementData.InitializeMeshMaterialData(ViewIfDynamicMeshCommand,  
                                        proxy, batch, staticMeshID,  
                                        false); //Not always false  
  
//Set our own parameters.  
element.MyFloat = deltaSeconds;  
element.MyVector = axis;  
//Now we can pass it into 'BuildMeshDrawCommands()'.
```

RDG Resources

Despite not being able to use a parameter struct for shaders, you should still make one to inform the RDG of your pass's inputs and outputs. I believe you could also manually tell the RDG what resources are used and set up render targets yourself through the RHI command-list, but that's more trouble than making the struct.

Unfortunately, as far as I can tell, **mesh shaders cannot have RDG texture/buffer parameters**. The reason is that there is no callback to set shader parameters right at the time of execution, after the RDG textures have a real RHI handle backing them. Your textures need to already have valid RHI handles when first setting up the pass.

Mesh Pass Processor

A Mesh Pass Processor, or MPP, generates the draw calls for your mesh pass given one mesh batch at a time. There is boilerplate to churn through, but the idea isn't

that complicated:

1. Override or shadow the function `AddMeshBatch(...)`
2. Compile your shader against the batch's Material and Vertex Factory
3. Pick draw settings like blend mode and depth usage
4. Ideally let the Material/batch override those draw settings
5. Create an “element data” instance (`FMeshMaterialShaderElementData` , mentioned above) to feed in your shader parameters
6. Generate the draw calls by invoking `BuildMeshDrawCommands(...)` .

The demo will use a very standard implementation which you can build off of.

Pass Execution

All that's left is to dispatch the mesh pass within your Scene-View Extension.



Call `AddSimpleMeshPass(...)` , provide the parameter struct (still needed to inform RDG) and various other details, and then inside the lambda you can create one or more of your mesh-pass processors and queue up some batches.

To actually get the mesh batches from the scene, as mentioned before you can execute my SVE helper function `ForEachComponent_RenderThread(...)` to iterate over

each pass component and its associated data — POD proxy, target primitive component, and the POD proxy *for* the primitive component. Inside this inner lambda, iterate through its batches with `EGP::ForEachBatch(...)`. Finally, inside *that* lambda, you can call your MPP's `AddMeshBatch(...)` implementation. See the demo for an example of this.

Demo

Let's add a mesh pass to the Game of Life effect. My demo scene has a variety of primitives in it, and here I give them interesting effect behavior through their Materials.

The static meshes emit living cells whenever they glow Blue

Niagara particles carve out trails of living cells on their orange side, but not on their black side.

The edges of the skinned meshes' heads emit living cells. The rest of the head destroys those cells.

To make the effects more obvious I've cleared out the Init pass in my effect material

so that the viewport starts with no living cells. I also removed the previous article's test behavior in the Simulate pass, which illustrated the behavior of different threshold values.

Landscapes :(

Unfortunately, while in theory all scene primitives can participate in this pass, I failed to get a landscape working. I had to write a special C++ helper just to find all `ULandscapeComponent` instances in the scene (they didn't belong directly to the Landscape actor, seemingly because of large-world-streaming stuff), and then they numbered in the hundreds which destroyed the performance of the custom pass.

If you want to dig deeper, I left in my C++/BP function which enumerates a landscape's multiple proxy components. Let me know if you can get landscapes working in your pass!

Mesh Pass Component

To start, let's define the component that marks a primitive as being part of the mesh pass. We will also define some per-primitive settings for our pass. For now the only parameter will be a Blend Mode enum, defining how the primitive affects the sim state texture: alpha-blending, additive blending, or multiply blending.

In the header, above the rest of the render pass, add this:

```
//The blend mode used for a primitive in the GoL mesh pass.
UENUM(BlueprintType)
enum class EGoLMeshBlendModes
{
    Alpha,
    Additive,
    Multiply,
    COUNT_UMETA(Hidden)
};
ENUM_RANGE_BY_COUNT(EGoLMeshBlendModes,
                    static_cast<int>(EGoLMeshBlendModes::COUNT));

//A POD struct containing all render parameters for one primitive
//    in our Game of Life effect.
USTRUCT(BlueprintType)
struct GOL_DEMO_API FGoLPrimitiveRenderSettings
```

```

{
    GENERATED_BODY()
public:

    UPROPERTY(BlueprintReadWrite, EditAnywhere)
    EGoLMeshBlendModes BlendMode = EGoLMeshBlendModes::Alpha;

    bool operator==(const FGoLPrimitiveRenderSettings& r2) const
    {
        return BlendMode == r2.BlendMode;
    }
};
inline uint32 GetTypeHash(const FGoLPrimitiveRenderSettings& r)
{
    //Dump all the fields into a tuple and let Unreal hash it for us.
    return GetTypeHash(MakeTuple(r.BlendMode));
}

//Marks a primitive-component (mesh, particle system, etc)
//    so that it renders into the GoL sim.
UCLASS(meta=(BlueprintSpawnableComponent))
class GOL_DEMO_API UGoLComponent : public U_EGP_RenderPassComponent
{
    GENERATED_BODY()
public:

    UPROPERTY(BlueprintReadWrite, EditAnywhere,
                meta=(ShowOnlyInnerProperties))
    FGoLPrimitiveRenderSettings RenderSettings;

    virtual TSubclassOf<U_EGP_RenderPass> GetPassType() const override;

    //The proxy struct is our own custom type,
    //    and we need to override some non-templated functions
    //    to create/destroy it within a byte buffer.
    //In most cases you can do this with the below helper macro.
    //The second argument is an R-value that creates the proxy.
    EGP_PASS_COMPONENT_SIMPLE_PROXY_IMPL(FGoLPrimitiveRenderSettings,
                                          RenderSettings)
};

```

Update the SVE's type signature to mention the component and its proxy:

```

struct GOL_DEMO_API F_GOL_PassSVE

```

```

: public T_EGP_RenderPassSceneViewExtension<U_GoL_RenderPass,
                                           UGoLComponent,
                                           FGoLPrimitiveRenderSettings>
...

```

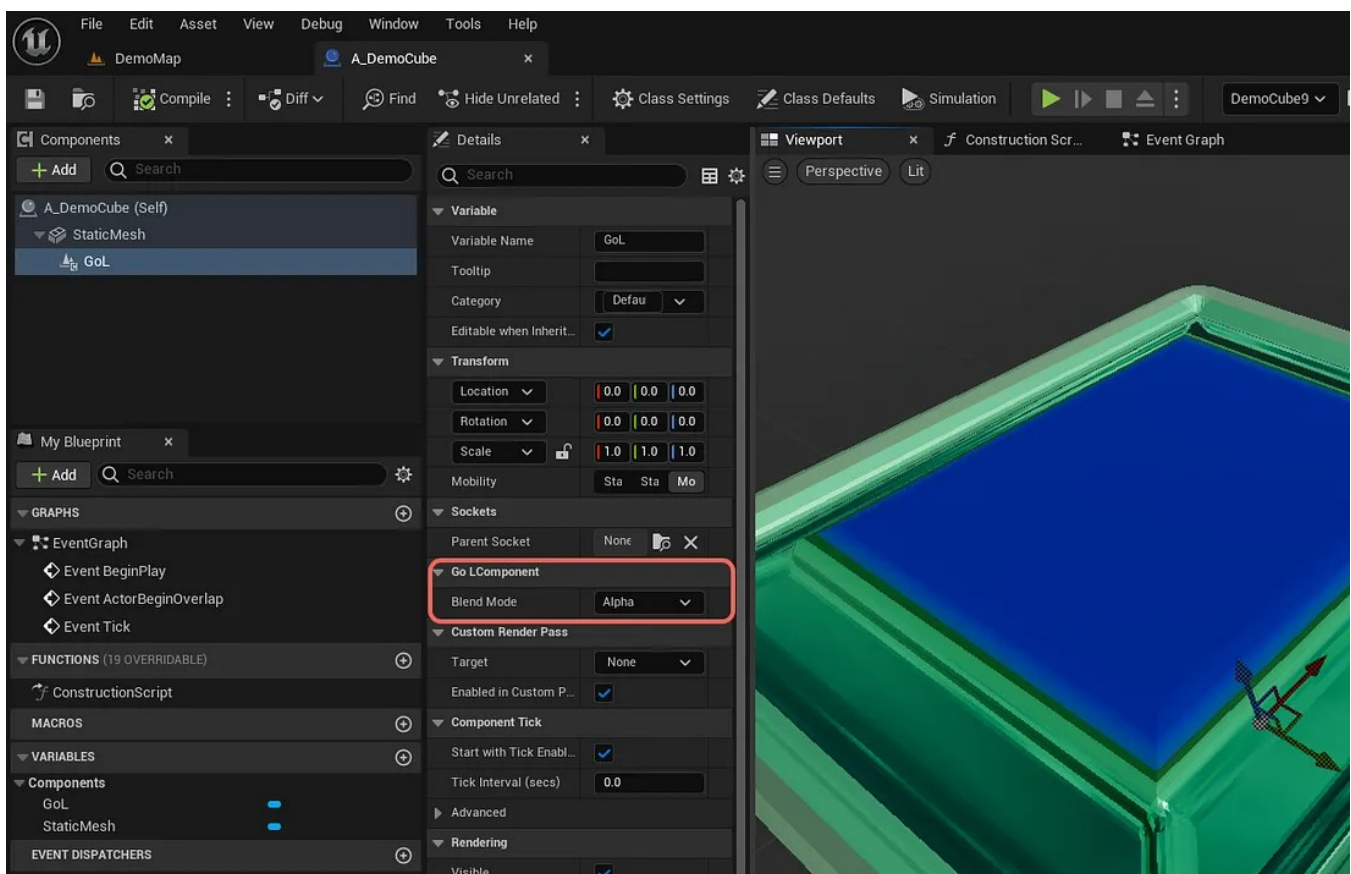
Implement functions in the cpp:

```

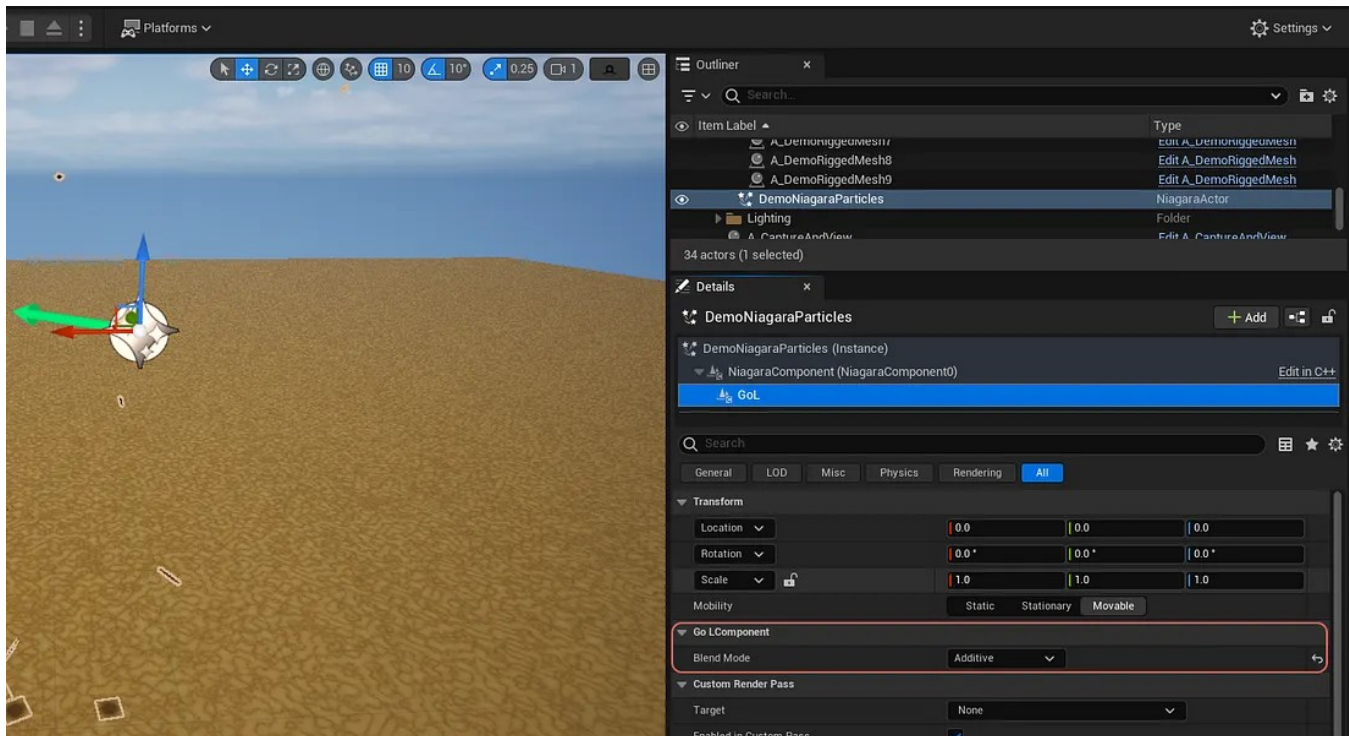
TSubclassOf<U_EGP_RenderPass> UGoLComponent::GetPassType() const
{
    return U_GoL_RenderPass::StaticClass();
}

```

Now go into your demo scene and attach this new component to any renderable thing you want to be in the pass.



The component on the static mesh, with its settings



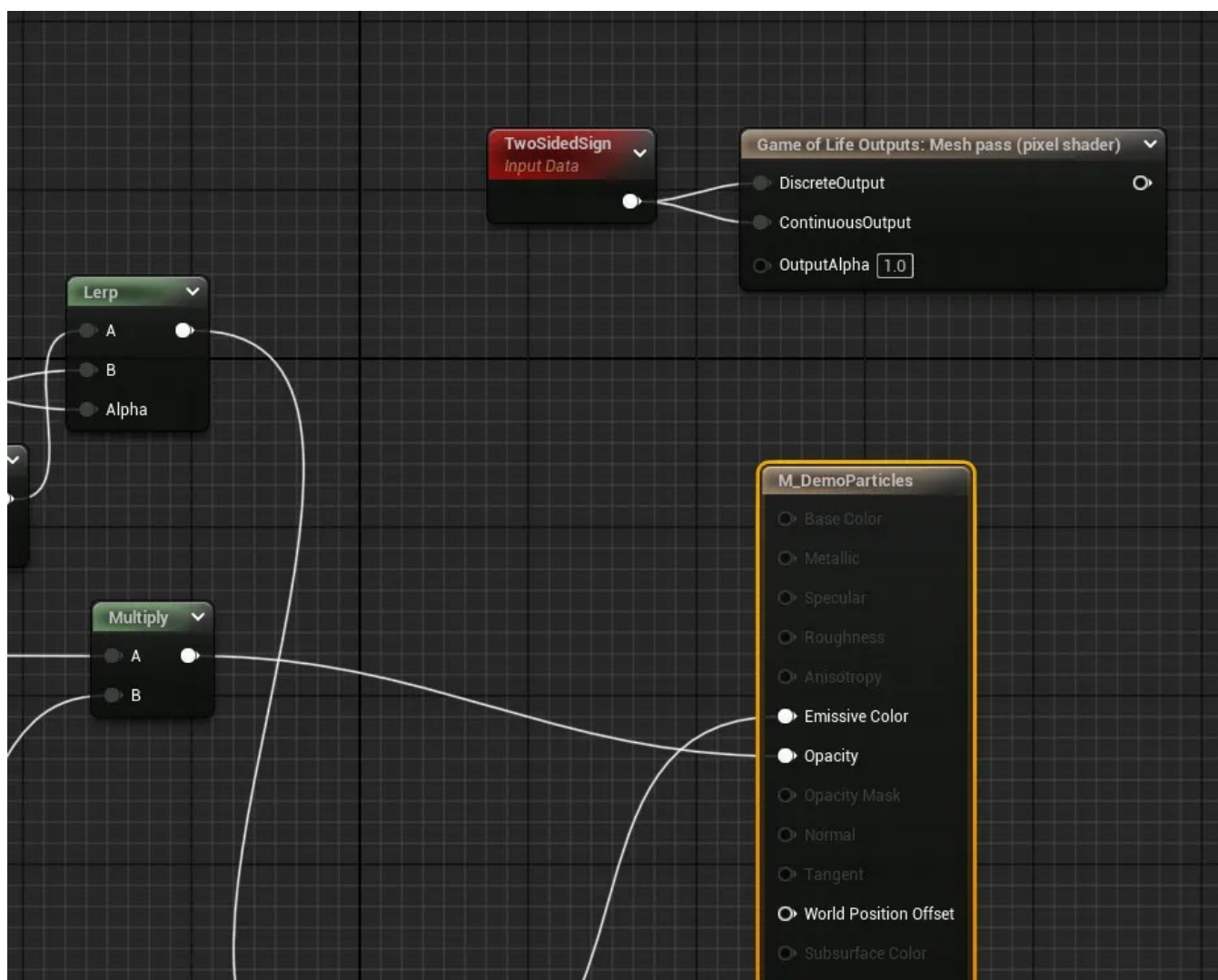
The component on the particle system, with its settings

New Material Outputs

Previously all our Material shaders were screen-space, using the Post-Process domain, and we had one uber-Material to define all pass behaviors. However the mesh pass will use the Materials already given to each primitive, which are in the Surface domain. We will define new Mesh-pass material output pins, which any 3D material can write to. Example Materials from my demo scene are below.



effect on the pass by feeding the Head mask into the GoL alpha.



The particle material outputs live cells if rendering its front-face, and dead cells otherwise. The Alpha output is useless because the particles use Additive blending.

Add the new node to a header file.

```
UCLASS(CollapseCategories, HideCategories=Object,
      DisplayName="GoL Outputs: Mesh")
class GOL_DEMO_API UMaterialExpressionGoLMeshOutputs : public UMaterialExpression
{
    GENERATED_BODY()
public:

    UPROPERTY(meta=(RequiredInput=false))
    FExpressionInput DiscreteOutput;
    UPROPERTY(meta=(RequiredInput=false))
    FExpressionInput ContinuousOutput;
```

```

UPROPERTY(meta=(RequiredInput=false))
FExpressionInput OutputAlpha;

UPROPERTY(EditAnywhere,
           meta=(OverridingInputProperty=DiscreteOutput))
float DiscreteOutputConst = 0.5f;
UPROPERTY(EditAnywhere,
           meta=(OverridingInputProperty=ContinuousOutput))
float ContinuousOutputConst = 0.5f;
UPROPERTY(EditAnywhere,
           meta=(OverridingInputProperty=OutputAlpha))
float OutputAlphaConst = 1.0f;

virtual FString GetFunctionName() const override {
    return TEXT("GoL_Outputs_Mesh_");
}
virtual FString GetDisplayName() const override {
    return TEXT("GoL Outputs: Mesh (pixel shader)");
}

#if WITH_EDITOR
virtual void GetCaption(TArray<FString>& output) const override {
    output.Add(TEXT("Game of Life Outputs: “
                    “Mesh pass (pixel shader)"));
}
virtual int32 GetNumOutputs() const override { return 3; }
virtual EShaderFrequency GetShaderFrequency() override {
    return SF_Pixel;
}
virtual int32 Compile(FMaterialCompiler*, int32 pinIdx) override;
#endif
};

```

Implement it in a cpp file.

```

int32 UMaterialExpressionGoLMeshOutputs::Compile(FMaterialCompiler* compiler,
                                                  int32 pinIdx)
{
    int32 codeID;
    auto doPin = [&](int32 i, FExpressionInput& pin, float* fallback)
    {
        if (pinIdx != i)
            return false;
        if (pin.IsConnected())

```

```

        codeID = pin.Compile(compiler);
    else if (fallback)
        codeID = compiler->Constant(*fallback);
    else
        codeID = INDEX_NONE;
    return true;
};

if (!doPin(0, DiscreteOutput, &DiscreteOutputConst) &&
    !doPin(1, ContinuousOutput, &ContinuousOutputConst) &&
    !doPin(2, OutputAlpha, &OutputAlphaConst))
{
    codeID = INDEX_NONE;
}
return compiler->CustomOutput(this, pinIdx, codeID);
}

```

Now you can add some initial outputs to your demo scene's Materials. However they won't do anything until we create and invoke the new shader.

Mesh-Material Shader

Start with the HLSL, in a new file *Shaders/Mesh.usf*:

```

#include "/Engine/Private/Common.usf"

//Include the Material code THEN the Vertex Factory code.
//For Intellisense, as a stand-in,
//    use the Material template and a specific built-in vertex factory.
#if defined(__RESHARPER__) || defined(__INTELLISENSE__)
    #define VERTEXSHADER 1
    #define PIXELSHADER 1
    #include "/Engine/Private/MaterialTemplate.usf"
    #include "/Engine/Private/LocalVertexFactory.usf"
#else
    #include "/Engine/Generated/Material.usf"
    #include "/Engine/Generated/VertexFactory.usf"
#endif

//Define the outputs from the VS into the PS.
struct FPassVSToPS
{
    float4 Position : SV_POSITION;

```

```
//Generated by the Vertex Factory
FVertexFactoryInterpolantsVSToPS VFInterpolants;
};

#if VERTEXSHADER
//The vertex shader is a pretty simple pass-through,
//  invoking the Vertex Factory for world-space data
//  and the Material for vertex animations.
void MainVS(FVertexFactoryInput inputs, out FPassVSToPS outputs)
{
    //Set up viewport uniforms.
    ResolvedView = ResolveView();

    //Invoke the vertex-factory to get input data.
    FVertexFactoryIntermediates vfIntermediates =
        GetVertexFactoryIntermediates(inputs);
    float4 worldPos4 = VertexFactoryGetWorldPosition(
        inputs,
        vfIntermediates
    );
    float3x3 tangentToLocal = VertexFactoryGetTangentToLocal(
        inputs,
        vfIntermediates
    );

    //Invoke the Material and apply its world-position-offset pin.
    FMaterialVertexParameters mParameters =
        GetMaterialVertexParameters(inputs, vfIntermediates,
                                     worldPos4.xyz, tangentToLocal);
    worldPos4.xyz += GetMaterialWorldPositionOffset(mParameters);

    //Calculate PS inputs.
    outputs.VFInterpolants = VertexFactoryGetInterpolantsVSToPS(
        inputs, vfIntermediates, mParameters
    );
    outputs.Position = INVARIANT(mul(
        VertexFactoryGetRasterizedWorldPosition(inputs,
                                                  vfIntermediates,
                                                  worldPos4),
        ResolvedView.TranslatedWorldToClip
    ));
}
#endif

#if PIXELSHADER

Texture2D<float2> PreviousStateTex;
SamplerState PreviousStateSampler;
```



```
//The pixel shader routes our custom Material output pins
//    to the simulation state.
void MainPS(in FPassVSToPS inputs //No comma; below macro may be empty
            OPTIONAL_IsFrontFace,
            //Output includes an Alpha channel for blending,
            //    even though the actual texture is two-channel
            out float4 outState : SV_Target0)
{
    //Process uniforms.
    #if INSTANCED_STEREO
        ResolvedView = ResolveView(input.FactoryInterpolants.EyeIndex);
    #else
        ResolvedView = ResolveView();
    #endif

    //Process Material and Vertex Factory inputs.
    FMaterialPixelParameters mParameters = GetMaterialPixelParameters(
        inputs.VFInterpolants, inputs.Position
    );
    FPixelMaterialInputs mInputs;
    CalcMaterialParameters(
        mParameters, mInputs,
        inputs.Position, bIsFrontFace
    );

    //Read and write our custom outputs.
    outState.x =
        #if HAVE_GoL_Outputs_Mesh_0
            GoL_Outputs_Mesh_0(mParameters)
        #else
            0
        #endif
    ;
    outState.y =
        #if HAVE_GoL_Outputs_Mesh_1
            GoL_Outputs_Mesh_1(mParameters)
        #else
            outState.x
        #endif
    ;
    outState.a =
        #if HAVE_GoL_Outputs_Mesh_2
            GoL_Outputs_Mesh_2(mParameters)
        #else
            1.0
        #endif
    ;
}
```

```

}

#endif

```

Now we can add a new class to the cpp file, along with an “element data” struct holdings its parameters. As mentioned before, we have to use the Legacy parameters here.

```

//Parameters for the mesh pass.
//Unfortunately we can't reference RDG resources here :(
struct FGoLMeshShaderElementData : public FMeshMaterialShaderElementData
{
    FRHITexture* PreviousStateTex;
    FRHISamplerState* PreviousStateSampler;
};

class FGoLMeshVS : public FMeshMaterialShader
{
public:
    DECLARE_SHADER_TYPE(FGoLMeshVS, MeshMaterial);

    static bool ShouldCompilePermutation(const FMeshMaterialShaderPermutationPa
    {
        return params.MaterialParameters.MaterialDomain == MD_Surface &&
            FMeshMaterialShader::ShouldCompilePermutation(params);
    }

    FGoLMeshVS() = default;
    FGoLMeshVS(const ShaderMetaType::CompiledShaderInitializerType& initializer
        : FMeshMaterialShader(initializer)
    {
    }
};

class FGoLMeshPS : public FMeshMaterialShader
{
public:
    DECLARE_SHADER_TYPE(FGoLMeshPS, MeshMaterial);

    static bool ShouldCompilePermutation(const FMeshMaterialShaderPermutationPa
    {
        return params.MaterialParameters.MaterialDomain == MD_Surface &&
            FMeshMaterialShader::ShouldCompilePermutation(params);
    }

```



```
}

FGoLMeshPS() = default;
FGoLMeshPS(const ShaderMetaType::CompiledShaderInitializerType& initializer
           : FMeshMaterialShader(initializer)
{
    //The user's Material may or may not make use of
    //    the previous state texture,
    //    so those uniforms must be marked Optional.
    PreviousStateTex.Bind(
        initializer.ParameterMap,
        TEXT("PreviousStateTex"),
        SPF_Optional
    );
    PreviousStateSampler.Bind(
        initializer.ParameterMap,
        TEXT("PreviousStateSampler"),
        SPF_Optional
    );
}

//Binds parameters to the shader for a given element.
void GetShaderBindings(
    const FScene* scene,
    ERHIFeatureLevel::Type featureLevel,
    const FPrimitiveSceneProxy* primitiveSceneProxy,
    const FMaterialRenderProxy& materialRenderProxy,
    const FMaterial& material,
    const FMeshPassProcessorRenderState& drawRenderState,
    const FGoLMeshShaderElementData& element,
    FMeshDrawSingleShaderBindings& shaderBindings) const
{
    FMeshMaterialShader::GetShaderBindings(
        scene, featureLevel,
        primitiveSceneProxy, materialRenderProxy, material,
        drawRenderState, element, shaderBindings
    );

    shaderBindings.AddTexture(PreviousStateTex, PreviousStateSampler,
                             element.PreviousStateSampler,
                             element.PreviousStateTex);
}

private:

    LAYOUT_FIELD(FShaderResourceParameter, PreviousStateTex);
    LAYOUT_FIELD(FShaderResourceParameter, PreviousStateSampler);
};
```

```

IMPLEMENT_MATERIAL_SHADER_TYPE(, FGoLMeshVS, TEXT("/GameOfLife/Mesh.usf"),
                               TEXT("MainVS"), SF_Vertex);
IMPLEMENT_MATERIAL_SHADER_TYPE(, FGoLMeshPS, TEXT("/GameOfLife/Mesh.usf"),
                               TEXT("MainPS"), SF_Pixel);

//This struct doesn't feed into either shader,
//    but tells the RDG about our mesh pass.
BEGIN_SHADER_PARAMETER_STRUCT(FGoLMeshPassParameters, )
    SHADER_PARAMETER_RDG_TEXTURE_SRV(Texture2D<float2>, PrevState)
    SHADER_PARAMETER_STRUCT_REF(FViewUniformShaderParameters, View)
    SHADER_PARAMETER_RDG_UNIFORM_BUFFER(FSceneUniformParameters, Scene)
    SHADER_PARAMETER_STRUCT_INCLUDE(FInstanceCullingDrawParams,
                                     InstanceCullingDrawParams)

    RENDER_TARGET_BINDING_SLOTS()
END_SHADER_PARAMETER_STRUCT()

```

Execute the Pass

Lastly we need to draw this pass in our Scene-View Extension, enlisting the help of a new Mesh Pass Processor.

Put this in your cpp file:

```

//New includes we'll need:
#include "Runtime/Renderer/Public/MeshPassProcessor.inl"
#include "SimpleMeshDrawCommandPass.h"
#include "EGP_DownsampleDepthPass.h"
#include "EGP_GetMeshBatches.h"

//Generates GoL draw calls for any kind of 3D primitive.
class FGoLMeshProcessor final : public FMeshPassProcessor
{
public:

    FMeshPassProcessorRenderState PassDrawState;

    FGoLMeshProcessor(const FScene* scene, const FSceneView* view,
                      ERHIFeatureLevel::Type featureLevel,
                      FMeshPassDrawListContext* commandsOutput,
                      FBlendStateRHIFRef blendState)
        : FMeshPassProcessor(scene, featureLevel, view, commandsOutput)
    {
        PassDrawState.SetBlendState(blendState);
    }

```

```
PassDrawState.SetDepthStencilState(
    TStaticDepthStencilState<false, CF_DepthNearOrEqual>::GetRHI()
);
}

void AddMeshBatch(const FMeshBatch& batch, uint64 batchElementMask,
    const FPrimitiveSceneProxy* proxy,
    int32 staticMeshID,
    FRHITexture* prevState,
    FRHISamplerState* prevStateSampler)
{
    //Get the first usable Material in the chain,
    //    starting at the batch's desired Material
    //    and ending at the Default Material.
    const FMaterialRenderProxy* fallbackMat = nullptr;
    const auto& resource =
        batch.MaterialRenderProxy->GetMaterialWithFallback(
            FeatureLevel, fallbackMat
        );
    const auto& materialProxy = fallbackMat ?
        *fallbackMat :
        *batch.MaterialRenderProxy;

    //Compile our shaders against
    //    the batch's Material and Vertex-Factory.
    TMeshProcessorShaders<FGoLMeshVS, FGoLMeshPS> shaderRefs;
    {
        FMaterialShaderTypes shaderTypes;
        shaderTypes.AddShaderType<FGoLMeshVS>();
        shaderTypes.AddShaderType<FGoLMeshPS>();

        FMaterialShaders materialShaders;
        verify(resource.TryGetShaders(
            shaderTypes,
            batch.VertexFactory->GetType(),
            materialShaders
        ));

        materialShaders.TryGetVertexShader(shaderRefs.VertexShader);
        materialShaders.TryGetPixelShader(shaderRefs.PixelShader);
    }

    //Configure per-element settings.
    FGoLMeshShaderElementData elementData;
    elementData.InitializeMeshMaterialData(
        ViewIfDynamicMeshCommand, proxy, batch, staticMeshID, false
    );
    //Set our shader parameters.
```

```

        elementData.PreviousStateTex = prevState;
        elementData.PreviousStateSampler = prevStateSampler;

        //Generate the draw calls for this batch.
        const FMeshDrawingPolicyOverrideSettings overrides =
            ComputeMeshOverrideSettings(batch);
        BuildMeshDrawCommands(
            batch, batchElementMask, proxy,
            materialProxy, resource, PassDrawState,
            MoveTemp(shaderRefs),
            ComputeMeshFillMode(resource, overrides),
            ComputeMeshCullMode(resource, overrides),
            FMeshDrawCommandSortKey::Default, EMeshPassFeatures::Default,
            elementData
        );
    }

    //The usual 'AddMeshBatch()' will not be used; instead
    //    we will use the above alternative with more parameters.
    virtual void AddMeshBatch(const FMeshBatch& batch,
                             uint64 batchElementMask,
                             const FPrimitiveSceneProxy* proxy,
                             int32 staticMeshID) override {
        check(false);
    }
};

```

Then go into the SVE's render callback, right after Simulate and before Display, and add this mesh pass:

```

//Draw our mesh pass into the sim state.
RDG_EVENT_SCOPE(graph, "GoL: Mesh passes (%i primitives)",
                Pass->GetComponentData_RenderThread().Num());
{

    FScene* renderScene = nullptr;
    if (view.Family != nullptr && view.Family->Scene != nullptr)
        renderScene = view.Family->Scene->GetRenderScene();

    //We want to use the scene's depth-texture for our mesh pass,
    //    however the Game of Life state texture exists on its own
    //    rather than being a subset of a larger render target,
    //    and also uses a lower resolution than the viewport.
    //To fix this, we need to resample the depth buffer.

```

```
FRDGTextureRef depthBuffer = inputs.SceneTextures->GetContents()
                                ->SceneDepthTexture;
if (view.ViewRect.Min != FIntPoint::ZeroValue ||
    depthBuffer->Desc.Extent != viewData.SimState->GetSizeXY())
{
    auto resampledDepthBufferDesc = depthBuffer->Desc;
    resampledDepthBufferDesc.Extent = viewData.SimState->GetSizeXY();
    FRDGTextureRef resampledDepthBuffer = graph.CreateTexture(
        resampledDepthBufferDesc, TEXT("GoL_ResampledSceneDepth")
    );

    EGP::AddDownsampleDepthPass(
        graph, view,
        FScreenPassTexture{ depthBuffer, view.ViewRect },
        FScreenPassRenderTarget{ resampledDepthBuffer,
                                ERenderTargetLoadAction::ENoAction },
        EDownsampleDepthFilter::Checkerboard
    );
    depthBuffer = resampledDepthBuffer;
}

auto nextSimStateRDG = RegisterExternalTexture(
    graph, viewData.SimBuffer,
    TEXT("GoL_NextState")
);

//Set up the RDG configuration of the pass.
auto* passParams = graph.AllocParameters<FGoLMeshPassParameters>();
passParams->View = view.ViewUniformBuffer;
passParams->Scene = GetSceneUniformBufferRef(graph, view);
passParams->PrevState =
    graph.CreateSRV(FRDGTextureSRVDesc{ simStateRDG });
passParams->RenderTargets[0] = FRenderTargetBinding{
    nextSimStateRDG,
    ERenderTargetLoadAction::ELoad
};
passParams->RenderTargets.DepthStencil = {
    depthBuffer,
    ERenderTargetLoadAction::ELoad,
    FExclusiveDepthStencil::DepthRead_StencilNop
};

//Dispatch the draw calls.
AddCopyTexturePass(graph, simStateRDG, nextSimStateRDG);
AddSimpleMeshPass(graph, passParams, renderScene, view, nullptr,
    RDG_EVENT_NAME("GoLMeshes"),
    FIntRect{ FIntPoint::ZeroValue,
        simStateRDG->Desc.Extent },
```

```

        [&](FDynamicPassMeshDrawListContext* output)
    {
        //Define one mesh processor for each blend mode.
        FGoLMeshProcessor meshProcessorAlpha{
            renderScene, &view, view.FeatureLevel, output,
            TStaticBlendState<CW_RGBA, BO_Add,
                BF_SourceAlpha, BF_InverseSourceAlpha
            >::GetRHI()
        };
        FGoLMeshProcessor meshProcessorAdditive{
            renderScene, &view, view.FeatureLevel, output,
            TStaticBlendState<CW_RGBA, BO_Add, BF_One, BF_One>::GetRHI()
        };
        FGoLMeshProcessor meshProcessorMultiply{
            renderScene, &view, view.FeatureLevel, output,
            TStaticBlendState<CW_RGBA, BO_Add,
                BF_DestColor, BF_Zero
            >::GetRHI()
        };

        //For every scene primitive tagged with our component...
        ForEachComponent_RenderThread([&](
            const UGoLComponent& component,
            const FGoLPrimitiveRenderSettings& componentSettings,
            const UPrimitiveComponent& primitive,
            const FPrimitiveSceneProxy& primitiveProxy
        ) {
            //Pick the appropriate MPP for our blend mode.
            FGoLMeshProcessor* componentProcessor;
            switch (componentSettings.BlendMode)
            {
                case EGoLMeshBlendModes::Alpha:
                    componentProcessor = &meshProcessorAlpha;
                    break;
                case EGoLMeshBlendModes::Additive:
                    componentProcessor = &meshProcessorAdditive;
                    break;
                case EGoLMeshBlendModes::Multiply:
                    componentProcessor = &meshProcessorMultiply;
                    break;
                default: check(false); return;
            }

            //... and for every renderable mesh-batch within that component...
            EGP::ForEachBatch(view, &primitiveProxy,
                [&](const FMeshBatch& batch, uint64 mask,
                    const auto* sceneProxy,
                    int staticMeshIdIfApplicable)

```

```
{
    //...draw that batch.
    componentProcessor->AddMeshBatch(
        batch, mask, sceneProxy, staticMeshIdIfApplicable,
        viewData.SimState,
        TStaticSamplerState<SF_Bilinear,
            AM_Clamp, AM_Clamp
        >::GetRHI());
    });
});
});

//Swap 'previous' and 'next' textures.
std::swap(viewData.SimBuffer, viewData.SimState);
simStateRDG = nextSimStateRDG;

}
```

And that's it! You should be able to see your 3D scene affect the Game of Life now. Congrats on making it to the end of this series!

Code

The mesh pass and Bonus Render Effect (BRE for short) is added in the *part/7* branch:

[heyx3/UnrealExtendedGraphicsProgrammingDemo at part/7](#)

Details

Unanswered Questions

Despite all my research there are many things I'm left wondering. If you know, please reach out to me (or comment on this article) so I can get these questions answered!

1. How can I iterate over all visible mesh batches for a given `FViewInfo`?
2. Is it feasible to include a Landscape in our custom mesh pass? How do I get the landscape components and iterate over their mesh batches?
3. Is it possible at all to bind RDG resources (textures, buffers) to a mesh-material shader and/or mesh pass processor?

4. Is there some way of managing external per-viewport data that I missed?
5. How do I fix the problem with my shaders not being initially compiled, causing the Initialize shader to run with the Default Material instead?
6. If any Epic graphics programmer is reading this, please for the love of God clean up your shaders! I get ~30,000 warnings when cooking, which makes it impossible to look for warnings in my own shaders.

```
UE::MeshPassUtils::Dispatch
```

It appears there's a small utility in the engine to dispatch mesh-material compute shaders without all the rigamarole of a Mesh-Pass Processor. It's only used in the engine for volumetric rendering, but looks very useful for anybody doing 3D compute work! You can find it in *Renderer/Public/MeshPassUtils.h*.

Mesh Pass Processors

You may notice that the mesh pass processor has a virtual `AddMeshBatch(...)`, which our demo did not use because we wanted to pass some custom parameters in. This virtual function will not be called anywhere in your mesh pass, and may as well not exist. I suspect it is vestigial, or only relevant for the official built-in passes.

The function `BuildMeshDrawCommands(...)`, which you call within `AddMeshBatch(...)`, is a template. However it is *not* defined anywhere in the class's header. It's contained in a *.inl* file, and you have to add `#include "Runtime/Renderer/Public/MeshPassProcessor.inl"` in your own cpp or you'll get a linker error about the function not having a definition. It's not unheard of to use *.inl* files in C++ to separate template implementations from the rest of a header, but I've never seen a setup that requires users to include the file themselves in every cpp that calls it! Usually it's included at the bottom of the header.

Unreal Engine 5

Graphics Programming

Shaders

Technical Art





Edit profile

Written by William Mishra-Manning

59 followers · 5 following

Graphics Programming, Procedural Generation

Responses (2)



William Mishra-Manning

What are your thoughts?



Yudan Chen

Oct 1, 2025



Thank you for documenting your knowledge so meticulously! However, I noticed there are no licensing terms attached to your ExtendedGraphicsProgramming project on Github. Are there plans to add this? I'd like to try custom mesh passes built with that plugin in a commercial project.



1



1 reply

[Reply](#)



Brian Truelove

Jul 7, 2025



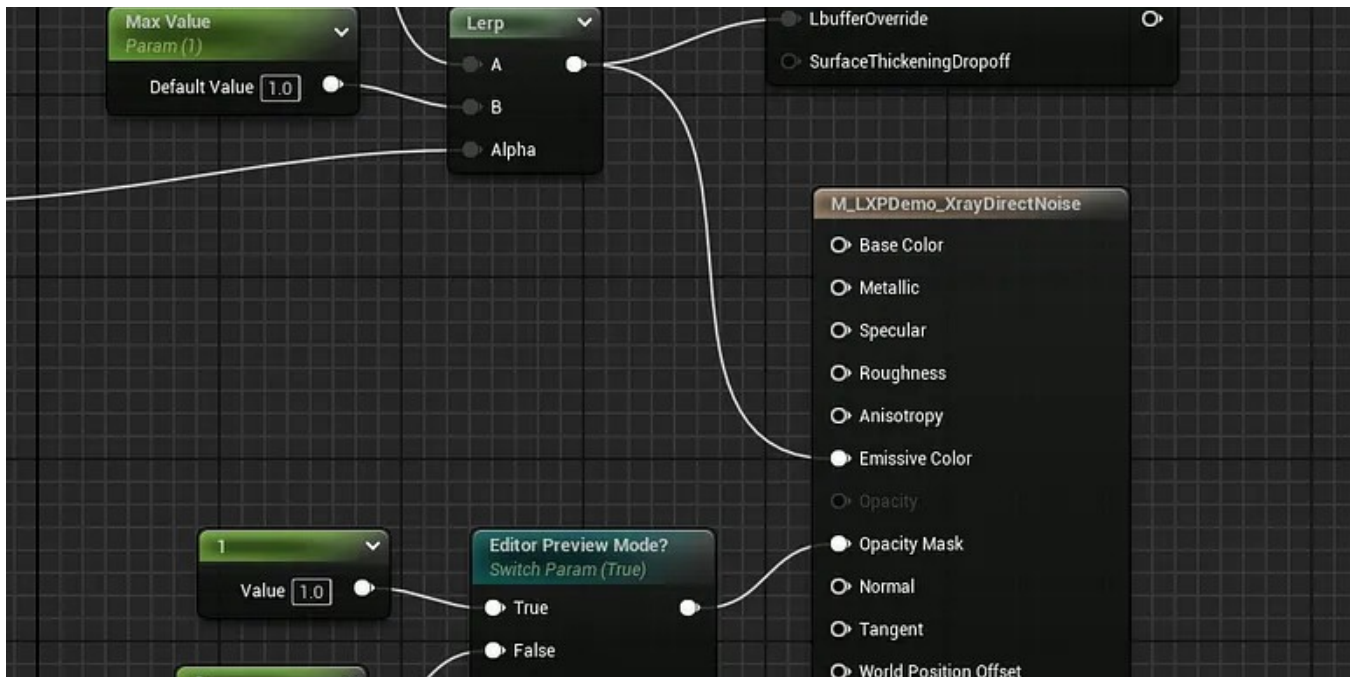
Thank you so much!



1

[Reply](#)

More from William Mishra-Manning



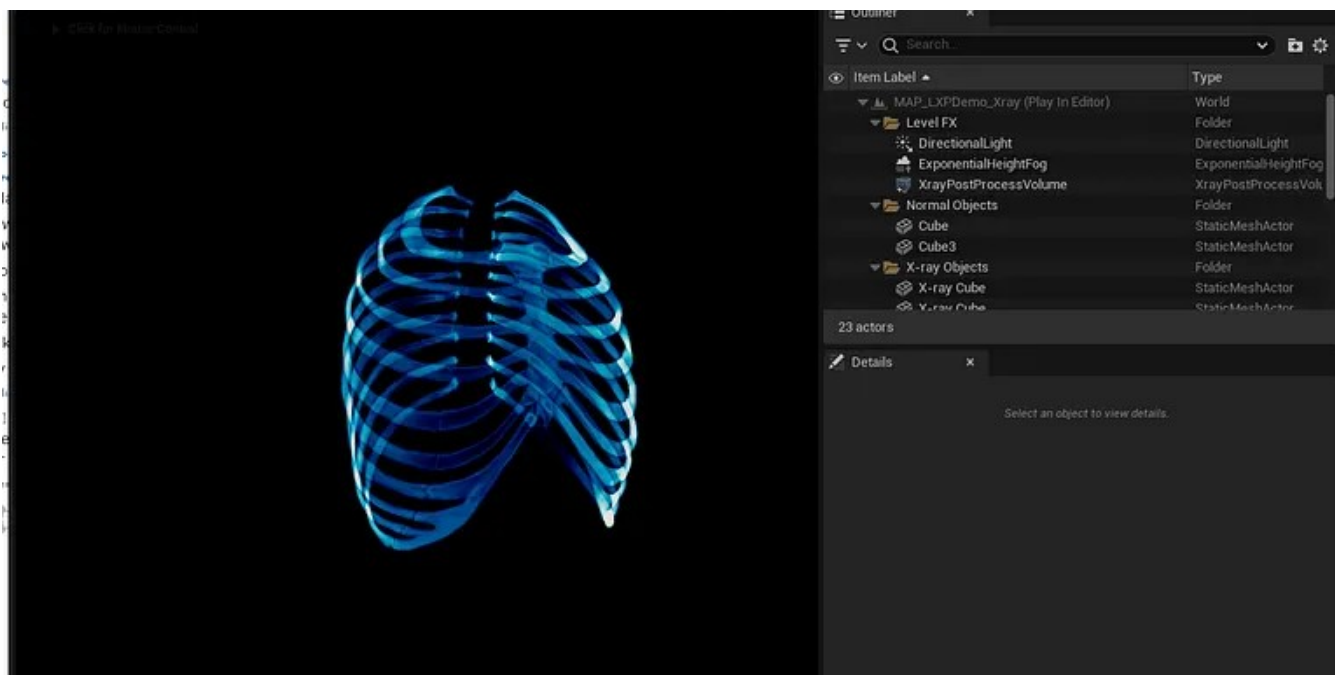
 William Mishra-Manning

Advanced Graphics Programming in Unreal, part 6

This is a seven-part article series.

Jun 25, 2025  1  2



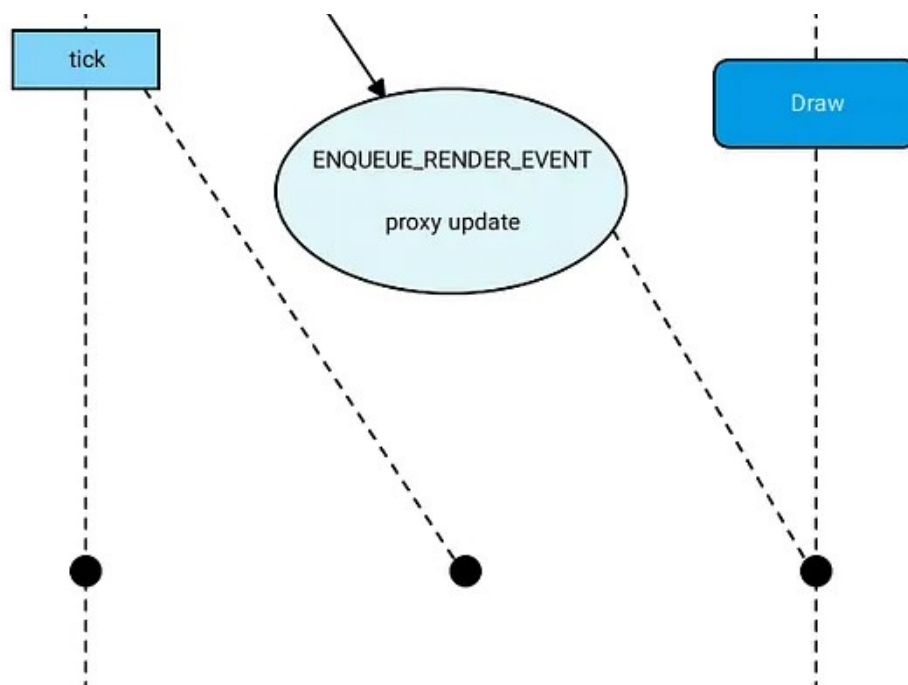


 William Mishra-Manning

Advanced Graphics Programming in Unreal, part 1

This is a seven-part article series.

Jun 24, 2025  31  1





William Mishra-Manning

Advanced Graphics Programming in Unreal, part 2

This is a seven-part article series.



William Mishra-Manning

Advanced Graphics Programming in Unreal, part 3

This is a seven-part article series.

Jun 24, 2025



6



See all from William Mishra-Manning

Recommended from Medium

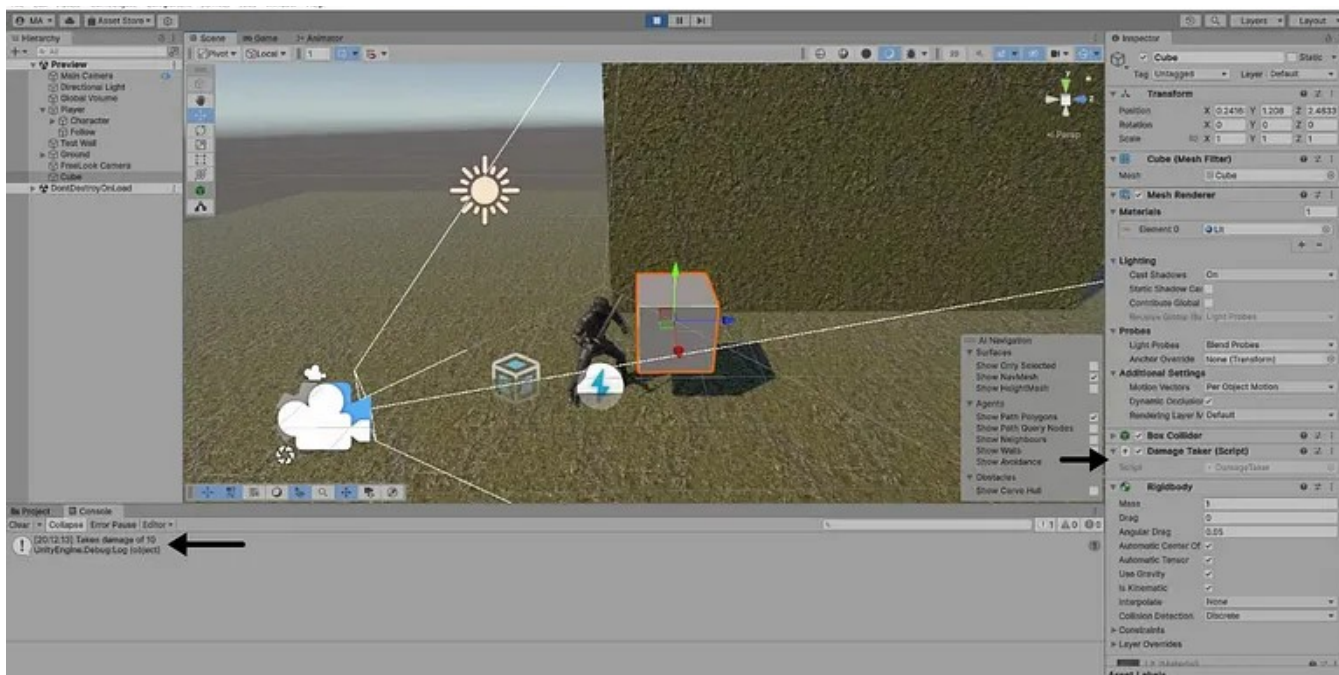


 Seulgie Han

C++ Move Semantics: From Copy to Efficient Resource Transfer

Source & Purpose: Based on Udacity C++ Nanodegree — Course3: Memory Management, this post is intended to help consolidate learning about...

Dec 13, 2025





Mahmoud Abaza

Building an RPG in Unity Part 3: Setting up camera and fighting system

In the previous chapter we explored how to add characters, animations and move them around into our game, but we have not given the camera...

Dec 16, 2025



9



Kastuvpokharel

QNetworkInformation (C++) vs NetworkInformation (QML): performance, efficiency, security and what...

If you want to know what QNetworkInformation is read this short article first, as this will be the perquisite for this article...

Dec 15, 2025





KM Kaloy Muñoz

Why CD Projekt Red Might Be the Last Prestige RPG Developer That Still Gets It

While the gaming industry shifts toward live-service models and diluted storytelling, CD Projekt Red continues to pursue ambitious...

Oct 23, 2025 6



Build & Deploy a Mobile Multiplayer Game with Unity — From Scratch to Android/iOS

Host

Client

Server

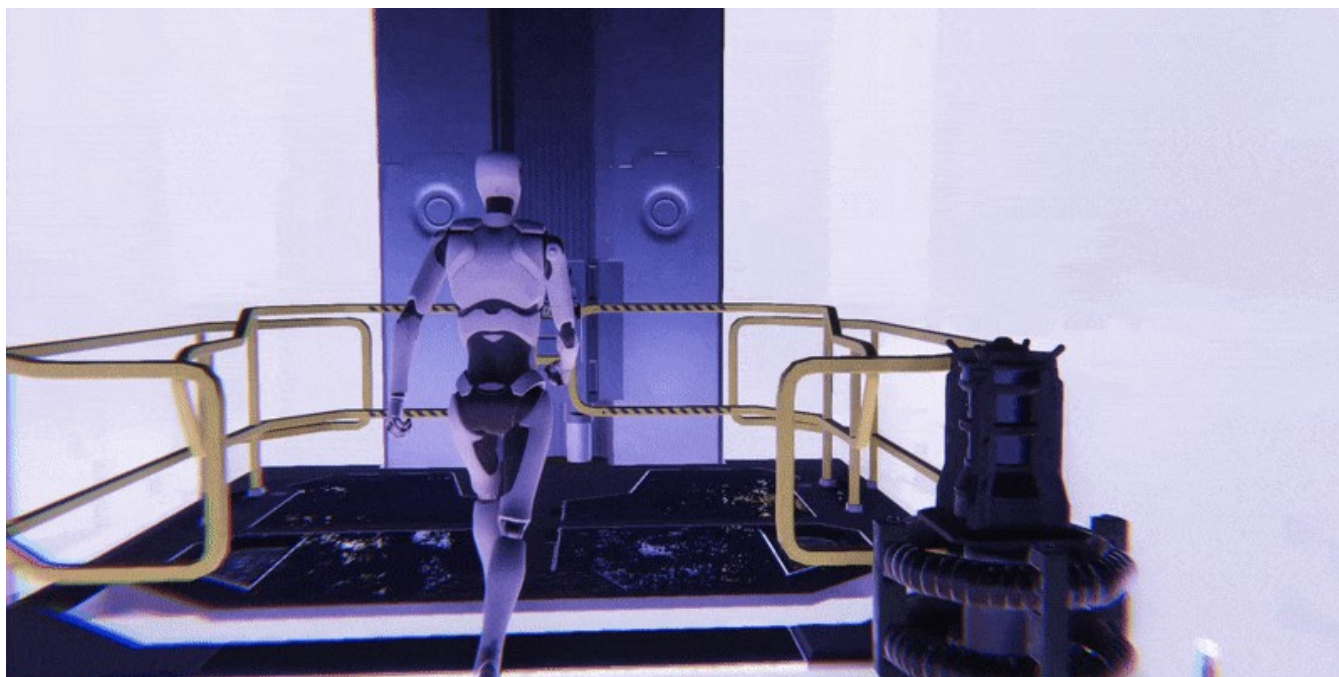


Tutorial Man

Build & Deploy a Mobile Multiplayer Game with Unity — From Scratch to Android/iOS

Read till last clap follow share

Nov 7, 2025 🖐️ 1



Jared Amlin

Elevator Mechanics in Unity

Elevators are a great addition to interior level design and let the player move between floors. Let's take a look at one way to make a...

Nov 7, 2025 🖐️ 5



See more recommendations